江苏文化传承与创新平台 - 源程序

```
// ========== FILE: ai-culture-design\crawler.mjs ==========
import fetch from 'node-fetch'
import { createHash } from 'crypto'
import { createWriteStream, existsSync, mkdirSync, readFileSync,
writeFileSync } from 'fs'
import { join, basename } from 'path'
import { fileURLToPath } from 'url'

const __dirname = fileURLToPath(new URL('.', import.meta.url))
const PUBLIC_DIR = join(__dirname, 'public')
const GALLERY_DIR = join(PUBLIC_DIR, 'gallery')
const HASH_FILE = join(GALLERY_DIR, '.hashes.json')
const PROGRESS_FILE = join(GALLERY_DIR, '.progress.json')

const MIN_WIDTH = 1920
const MIN_HEIGHT = 1080
const MAX_CONCURRENT = 3
const REQUEST_DELAY = 1500
const TIMEOUT = 30000
const MAX_RETRIES = 3

const JIANGSU_SCENIC_SPOTS = [
  { name: '苏州拙政园', nameEn: 'Suzhou Humble Administrator Garden',
category: 'heritage', tags: ['古典园林', '苏州', '世界遗产'] },
  { name: '南京中山陵', nameEn: 'Nanjing Sun Yat-sen Mausoleum',
category: 'heritage', tags: ['南京', '中山陵', '历史遗迹'] },
  { name: '太湖风光', nameEn: 'Taihu Lake scenery', category:
'landscape', tags: ['太湖', '无锡', '自然风光'] },
  { name: '扬州瘦西湖', nameEn: 'Yangzhou Slender West Lake',
category: 'heritage', tags: ['扬州', '瘦西湖', '江南水乡'] },
  { name: '周庄古镇', nameEn: 'Zhouzhuang water town', category:
'heritage', tags: ['周庄', '古镇', '江南水乡'] },
  { name: '南京夫子庙秦淮河', nameEn: 'Nanjing Confucius Temple
Qinhuai River', category: 'heritage', tags: ['南京', '夫子庙', '秦淮
河'] },
  { name: '连云港花果山', nameEn: 'Lianyungang Huaguo Mountain',
category: 'landscape', tags: ['连云港', '花果山', '自然风光'] },
  { name: '南京长江大桥', nameEn: 'Nanjing Yangtze River Bridge',
category: 'heritage', tags: ['南京', '长江大桥', '现代建筑'] },
  { name: '苏州博物馆', nameEn: 'Suzhou Museum I.M.Pei', category:
'heritage', tags: ['苏州', '博物馆', '现代建筑'] },
  { name: '南京明孝陵', nameEn: 'Nanjing Ming Xiaoling Tomb',
category: 'heritage', tags: ['南京', '明孝陵', '世界遗产'] },
```

```
  { name: '南通濠河夜景', nameEn: 'Nantong Haohe River night view',
category: 'landscape', tags: ['南通', '濠河', '夜景'] },
  { name: '镇江金山寺', nameEn: 'Zhenjiang Jinshan Temple', category:
'heritage', tags: ['镇江', '金山寺', '佛教文化'] },
  { name: '盐城丹顶鹤湿地', nameEn: 'Yancheng red crowned crane
wetland', category: 'landscape', tags: ['盐城', '湿地', '自然风光
'] },
  { name: '无锡鼋头渚', nameEn: 'Wuxi Yuantouzhu Turtle Head Isle',
category: 'landscape', tags: ['无锡', '鼋头渚', '太湖'] },
  { name: '苏州虎丘塔', nameEn: 'Suzhou Huqiu Tiger Hill Pagoda',
category: 'heritage', tags: ['苏州', '虎丘', '历史遗迹'] },
  { name: '南京玄武湖', nameEn: 'Nanjing Xuanwu Lake', category:
'landscape', tags: ['南京', '玄武湖', '城市公园'] },
  { name: '常州天目湖', nameEn: 'Changzhou Tianmu Lake', category:
'landscape', tags: ['常州', '天目湖', '自然风光'] },
  { name: '徐州云龙湖', nameEn: 'Xuzhou Yunlong Lake', category:
'landscape', tags: ['徐州', '云龙湖', '城市风光'] },
  { name: '苏州同里古镇', nameEn: 'Suzhou Tongli water town',
category: 'heritage', tags: ['同里', '古镇', '江南水乡'] },
  { name: '南京栖霞山', nameEn: 'Nanjing Qixia Mountain autumn',
category: 'landscape', tags: ['南京', '栖霞山', '红叶'] }
]

const IMAGE_SOURCES = [
  {
    name: 'Unsplash',
    baseUrl: 'https://api.unsplash.com/search/photos',
    needsAuth: true,
    buildUrl: (spot, page = 1) => {
      const accessKey = process.env.UNSPLASH_ACCESS_KEY || ''
      if (!accessKey) return null
      return `${spot.baseUrl}?query=${encodeURIComponent(spot.nameEn
+ ' Jiangsu
China')}&per_page=5&page=${page}&orientation=landscape&client_id=${ac
cessKey}`
    },
    parseResponse: (data) => {
      if (!data.results) return []
      return data.results
        .filter(r => r.width >= MIN_WIDTH && r.height >= MIN_HEIGHT)
        .map(r => ({
          url: r.urls?.raw ? `${r.urls.raw}&w=1920&q=85` :
r.urls?.regular,
          width: r.width,
```

```
          height: r.height,
          source: 'unsplash',
          sourceUrl: r.links?.html,
          author: r.user?.name,
          authorUrl: r.user?.links?.html
        }))
    }
  },
  {
    name: 'Pexels',
    baseUrl: 'https://api.pexels.com/v1/search',
    needsAuth: true,
    buildUrl: (spot, page = 1) => {
      const apiKey = process.env.PEXELS_API_KEY || ''
      if (!apiKey) return null
      return `${spot.baseUrl}?query=${encodeURIComponent(spot.nameEn
+ ' Jiangsu China')}&per_page=5&page=${page}&orientation=landscape`,
      headers: { Authorization: apiKey }
    },
    parseResponse: (data) => {
      if (!data.photos) return []
      return data.photos
        .filter(p => p.width >= MIN_WIDTH && p.height >= MIN_HEIGHT)
        .map(p => ({
          url: p.src?.landscape || p.src?.original,
          width: p.width,
          height: p.height,
          source: 'pexels',
          sourceUrl: p.url,
          author: p.photographer,
          authorUrl: p.photographer_url
        }))
    }
  },
  {
    name: 'Pixabay',
    baseUrl: 'https://pixabay.com/api/',
    needsAuth: true,
    buildUrl: (spot, page = 1) => {
      const apiKey = process.env.PIXABAY_API_KEY || ''
      if (!apiKey) return null
      return
`${spot.baseUrl}?key=${apiKey}&q=${encodeURIComponent(spot.nameEn + '
Jiangsu
```

```
China')}&image_type=photo&orientation=horizontal&min_width=${MIN_WIDT
H}&min_height=${MIN_HEIGHT}&per_page=5&page=${page}&safesearch=true`
    },
    parseResponse: (data) => {
      if (!data.hits) return []
      return data.hits.map(h => ({
        url: h.largeImageURL || h.webformatURL?.replace('_640',
'_1280'),
        width: h.imageWidth,
        height: h.imageHeight,
        source: 'pixabay',
        sourceUrl: h.pageURL,
        author: h.user,
        authorUrl: `https://pixabay.com/users/${h.user}-
${h.user_id}/`
      }))
    }
  }
]

class JiangsuImageCrawler {
  constructor() {
    this.downloadedHashes = this.loadHashes()
    this.progress = this.loadProgress()
    this.stats = { total: 0, downloaded: 0, skipped: 0, failed: 0 }
  }

  loadHashes() {
    try {
      if (existsSync(HASH_FILE)) {
        return JSON.parse(readFileSync(HASH_FILE, 'utf-8'))
      }
    } catch (e) {
      console.log('⚠  无法加载哈希记录，将创建新记录')
    }
    return {}
  }

  saveHashes() {
    writeFileSync(HASH_FILE, JSON.stringify(this.downloadedHashes,
null, 2))
  }

  loadProgress() {
```

```
    try {
      if (existsSync(PROGRESS_FILE)) {
        return JSON.parse(readFileSync(PROGRESS_FILE, 'utf-8'))
      }
    } catch (e) {}
    return { completed: [], lastSpotIndex: 0 }
  }

  saveProgress() {
    writeFileSync(PROGRESS_FILE, JSON.stringify(this.progress, null,
2))
  }

  async calculateHash(buffer) {
    return createHash('md5').update(buffer).digest('hex')
  }

  isDuplicate(hash) {
    return !!this.downloadedHashes[hash]
  }

  delay(ms) {
    return new Promise(resolve => setTimeout(resolve, ms))
  }

  printProgress(spot, current, total) {
    const percent = Math.round((current / total) * 100)
    const bar = '■'.repeat(Math.floor(percent / 2)) + '░'.repeat(50
- Math.floor(percent / 2))
    process.stdout.write(`\r📋 进度: [${bar}] ${percent}% |
${current}/${total} | 当前: ${spot.name} |
☑${this.stats.downloaded} ⏭${this.stats.skipped}
❌${this.stats.failed}`)
  }

  async fetchWithRetry(url, options = {}, retries = MAX_RETRIES) {
    for (let i = 0; i < retries; i++) {
      try {
        const controller = new AbortController()
        const timeoutId = setTimeout(() => controller.abort(),
TIMEOUT)
        const response = await fetch(url, { ...options, signal:
controller.signal })
        clearTimeout(timeoutId)
```

```
        if (!response.ok) throw new Error(`HTTP ${response.status}:
${response.statusText}`)
        return response
      } catch (error) {
        if (i === retries - 1) throw error
        console.log(`\n🔄 重试 (${i + 1}/${retries}):
${error.message}`)
        await this.delay(2000 * (i + 1))
      }
    }
  }

  async searchImages(spot) {
    const results = []
    for (const source of IMAGE_SOURCES) {
      try {
        const url = source.buildUrl(spot)
        if (!url) continue
        const headers = source.buildUrl.headers || {}
        const response = await this.fetchWithRetry(url, { headers })
        const data = await response.json()
        const parsed = source.parseResponse(data)
        results.push(...parsed)
        await this.delay(REQUEST_DELAY)
      } catch (error) {
        console.log(`\n⚠️  ${source.name} 搜索失败 (${spot.name}):
${error.message}`)
      }
    }
    return results
  }

  async downloadImage(imageInfo, spot) {
    try {
      const response = await this.fetchWithRetry(imageInfo.url)
      const buffer = Buffer.from(await response.arrayBuffer())
      const hash = await this.calculateHash(buffer)

      if (this.isDuplicate(hash)) {
        this.stats.skipped++
        return null
      }
```

```
      const fileName =
`jiangsu_${spot.nameEn.toLowerCase().replace(/[^a-z0-9]+/g,
'_')}_${Date.now()}.jpg`
      const filePath = join(GALLERY_DIR, fileName)

      const writeStream = createWriteStream(filePath)
      writeStream.write(buffer)
      writeStream.end()

      await new Promise((resolve, reject) => {
        writeStream.on('finish', resolve)
        writeStream.on('error', reject)
      })

      this.downloadedHashes[hash] = {
        fileName,
        spot: spot.name,
        source: imageInfo.source,
        author: imageInfo.author,
        downloadedAt: new Date().toISOString()
      }
      this.saveHashes()

      this.stats.downloaded++
      return {
        fileName,
        spot,
        imageInfo,
        hash
      }
    } catch (error) {
      this.stats.failed++
      console.log(`\n❌ 下载失败: ${error.message}`)
      return null
    }
  }

  async crawl() {
    console.log('🚀 江苏风景图片爬虫启动')
    console.log(`📁 图片保存目录: ${GALLERY_DIR}`)
    console.log(`📐 最低分辨率要求: ${MIN_WIDTH}×${MIN_HEIGHT}`)
    console.log(`🏞️  目标景点数量: ${JIANGSU_SCENIC_SPOTS.length}`)
    console.log('═'.repeat(60))
```

```
    if (!existsSync(GALLERY_DIR)) {
      mkdirSync(GALLERY_DIR, { recursive: true })
    }

    const results = []
    const totalSpots = JIANGSU_SCENIC_SPOTS.length

    for (let i = 0; i < totalSpots; i++) {
      const spot = JIANGSU_SCENIC_SPOTS[i]
      this.printProgress(spot, i + 1, totalSpots)

      if (this.progress.completed.includes(spot.name)) {
        this.stats.skipped++
        continue
      }

      try {
        const images = await this.searchImages(spot)
        this.stats.total += images.length

        if (images.length > 0) {
          const bestImage = images.sort((a, b) => (b.width *
b.height) - (a.width * a.height))[0]
          const result = await this.downloadImage(bestImage, spot)
          if (result) {
            results.push(result)
          }
        } else {
          console.log(`\n⚠  未找到 ${spot.name} 的高清图片`)
        }

        this.progress.completed.push(spot.name)
        this.saveProgress()
      } catch (error) {
        console.log(`\n❌ 处理 ${spot.name} 时出错:
${error.message}`)
      }

      await this.delay(REQUEST_DELAY)
    }

    console.log('\n\n' + '═'.repeat(60))
    console.log('📊 爬取结果统计:')
    console.log(`   总搜索图片: ${this.stats.total}`)
```

```
    console.log(`   成功下载: ${this.stats.downloaded}`)
    console.log(`   跳过(重复): ${this.stats.skipped}`)
    console.log(`   失败: ${this.stats.failed}`)
    console.log('═'.repeat(60))

    this.generateGalleryData(results)
    return results
  }

  generateGalleryData(results) {
    const galleryData = results.map((result, index) => {
      const spot = result.spot
      const imageInfo = result.imageInfo
      return {
        id: index + 1,
        title: spot.name,
        description: `${spot.name}是江苏省著名景点，展现了江苏独特的
自然风光和人文魅力。`,
        imageUrl: `/gallery/${result.fileName}`,
        author: imageInfo.author || '江苏风光',
        authorAvatar: `https://i.pravatar.cc/150?img=${(index % 70) +
1}`,
        category: spot.category,
        tags: spot.tags,
        likes: Math.floor(Math.random() * 300) + 50,
        views: Math.floor(Math.random() * 5000) + 500,
        collects: Math.floor(Math.random() * 100) + 10,
        isLiked: false,
        isCollected: false,
        isFollowing: false,
        isPublic: true,
        allowDownload: true,
        createTime: new Date().toISOString().split('T')[0],
        imageHeight: 180 + Math.floor(Math.random() * 100),
        comments: [],
        source: imageInfo.source,
        sourceUrl: imageInfo.sourceUrl,
        photoAuthor: imageInfo.author,
        photoAuthorUrl: imageInfo.authorUrl
      }
    })

    const dataFilePath = join(GALLERY_DIR, 'gallery-data.json')
    writeFileSync(dataFilePath, JSON.stringify(galleryData, null, 2))
```

```
    console.log(`\n📄 画廊数据已生成: ${dataFilePath}`)
  }
}

async function main() {
  const args = process.argv.slice(2)
  const command = args[0]

  if (command === 'clean') {
    console.log('🧹 清理已下载的图片和记录...')
    if (existsSync(GALLERY_DIR)) {
      const { rmSync } = await import('fs')
      rmSync(GALLERY_DIR, { recursive: true, force: true })
      console.log('☑ 清理完成')
    }
    return
  }

  if (command === 'status') {
    console.log('📊 当前爬取状态:')
    if (existsSync(HASH_FILE)) {
      const hashes = JSON.parse(readFileSync(HASH_FILE, 'utf-8'))
      console.log(`   已下载图片: ${Object.keys(hashes).length}`)
      Object.values(hashes).forEach(h => {
        console.log(`   - ${h.spot} (${h.fileName}) [来源:
${h.source}]`)
      })
    } else {
      console.log('   尚未下载任何图片')
    }
    if (existsSync(PROGRESS_FILE)) {
      const progress = JSON.parse(readFileSync(PROGRESS_FILE, 'utf-
8'))
      console.log(`   已完成景点:
${progress.completed.length}/${JIANGSU_SCENIC_SPOTS.length}`)
    }
    return
  }

  const crawler = new JiangsuImageCrawler()
  await crawler.crawl()
}

main().catch(console.error)
```

```
// ========== FILE: ai-culture-design\index.html ==========
<!doctype html>
<html lang="zh-CN">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="./favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <meta name="description" content="江苏文化地图 - 探索江苏丰富的文
化遗产" />
    <title>江苏文化地图</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>

// ========== FILE: ai-culture-design\server.js ==========
import http from 'http';
import fs from 'fs';
import path from 'path';

const server = http.createServer((req, res) => {
  let filePath = '.' + req.url;
  if (filePath === './') {
    filePath = './index.html';
  }

  const extname = String(path.extname(filePath)).toLowerCase();
  const mimeTypes = {
    '.html': 'text/html',
    '.js': 'text/javascript',
    '.css': 'text/css',
    '.json': 'application/json',
    '.png': 'image/png',
    '.jpg': 'image/jpg',
    '.gif': 'image/gif',
    '.svg': 'image/svg+xml',
    '.wav': 'audio/wav',
    '.mp4': 'video/mp4',
    '.woff': 'application/font-woff',
    '.ttf': 'application/font-ttf',
```

```
    '.eot': 'application/vnd.ms-fontobject',
    '.otf': 'application/font-otf',
    '.wasm': 'application/wasm'
  };

  const contentType = mimeTypes[extname] || 'application/octet-
stream';

  fs.readFile(path.join(__dirname, 'dist', filePath), (error,
content) => {
    if (error) {
      if (error.code == 'ENOENT') {
        fs.readFile(path.join(__dirname, 'dist', 'index.html'),
(error, content) => {
          res.writeHead(200, { 'Content-Type': 'text/html' });
          res.end(content, 'utf-8');
        });
      } else {
        res.writeHead(500);
        res.end('Sorry, check with the site admin for error: ' +
error.code + ' ..\n');
        res.end();
      }
    } else {
      res.writeHead(200, { 'Content-Type': contentType });
      res.end(content, 'utf-8');
    }
  });
});

const PORT = process.env.PORT || 8000;
server.listen(PORT, '127.0.0.1', () => {
  console.log(`Server running at http://127.0.0.1:${PORT}/`);
});

// ========== FILE: ai-culture-design\src\AI-API\deepseek.js
==========
// DeepSeek API 包装函数

// 生成文化设计内容
export async function generateCultureDesign(designType,
culturalElements, style, purpose) {
  try {
    // 模拟 API 调用延迟
```

```
    await new Promise(resolve => setTimeout(resolve, 1000));

    // 根据设计类型和文化元素生成内容
    let generatedContent = '';

    switch (designType) {
      case '海报设计':
        generatedContent = `根据${culturalElements}文化元素，设计了一款${style}风格的海报，适合用于${purpose}。设计融入了传统元素与现代美学，色彩搭配协调，视觉冲击力强。`;
        break;
      case '视频创意':
        generatedContent = `基于${culturalElements}文化元素，创作了一个${style}风格的视频创意方案，适合${purpose}使用。视频包含开场、主题展示、细节特写和结尾升华四个部分，节奏明快，情感真挚。`;
        break;
      case '文案策划':
        generatedContent = `围绕${culturalElements}文化元素，撰写了一份${style}风格的文案，适用于${purpose}。文案结构清晰，语言优美，充分展现了文化内涵和品牌价值。`;
        break;
      case '品牌设计':
        generatedContent = `结合${culturalElements}文化元素，设计了一套${style}风格的品牌视觉系统，适合${purpose}使用。包括标志、色彩系统、字体选择和应用规范，整体风格统一且富有文化底蕴。`;
        break;
      default:
        generatedContent = `基于${culturalElements}文化元素，创作了${style}风格的设计方案，适用于${purpose}。设计充分体现了文化特色，同时满足现代审美需求。`;
    }

    return {
      success: true,
      content: generatedContent,
      elements: culturalElements,
      style: style,
      purpose: purpose
    };
  } catch (error) {
    console.error('DeepSeek API 调用失败:', error);
    // 返回备用响应
    return {
      success: false,
```

```
      content: `基于${culturalElements}文化元素的设计方案正在生成中，这是一个${style}风格的设计，适用于${purpose}。`,
      elements: culturalElements,
      style: style,
      purpose: purpose
    };
  }
}

// 生成文化相关的创意内容
export async function generateCreativeContent(prompt, culturalContext) {
  try {
    // 模拟 API 调用延迟
    await new Promise(resolve => setTimeout(resolve, 800));

    return {
      success: true,
      content: `根据您的需求，结合${culturalContext}文化背景，生成了以下创意内容：\n\n${prompt}\n\n这份内容融入了传统元素，同时展现了现代创意视角。`
    };
  } catch (error) {
    console.error('DeepSeek 创意内容生成失败:', error);
    return {
      success: false,
      content: `创意内容生成中，基于${culturalContext}文化背景和您的需求，我们正在为您打造独特的创意方案。`
    };
  }
}

// 分析文化元素
export async function analyzeCulturalElements(elements) {
  try {
    // 模拟 API 调用延迟
    await new Promise(resolve => setTimeout(resolve, 600));

    return {
      success: true,
      analysis: `对${elements}文化元素的分析：这些元素代表了丰富的文化内涵，包括历史传统、艺术表现形式和社会价值观念。它们可以通过现代设计手法重新诠释，创造出既有文化底蕴又符合当代审美的作品。`
    };
```

```
  } catch (error) {
    console.error('文化元素分析失败:', error);
    return {
      success: false,
      analysis: `正在分析${elements}文化元素，我们将为您提供详细的文化背景和设计建议。`
    };
  }
}

// ========== FILE: ai-culture-design\src\AI-API\index.js ==========
// 导出所有 API 相关功能
export * from './volcengine.js';

// ========== FILE: ai-culture-design\src\AI-API\volcengine.js ==========
// 火山引擎图像生成 API 调用封装

const VOLCENGINE_API_KEY = 'cdec8ece-fa6d-4fad-b9a6-6a2d3b0b6143';
const VOLCENGINE_API_URL = 'https://ark.cn-beijing.volces.com/api/v3/images/generations';

const MIN_PIXELS = 3686400;
const VALID_SIZES = ['2048x2048', '1920x1920', '2560x1440', '1440x2560', '1920x1088', '1088x1920', '2304x1728', '1728x2304'];

function getValidSize(size) {
  if (VALID_SIZES.includes(size)) {
    return size;
  }
  const sizeMap = {
    '1K': '2048x2048',
    '2K': '2048x2048',
    '高清': '2048x2048',
    '竖版': '1440x2560',
    '横版': '2560x1440',
    '手机壁纸': '1088x1920',
    '电脑壁纸': '1920x1088',
    '方形': '2048x2048'
  };
  return sizeMap[size] || '2048x2048';
}
```

```
export async function generateImageWithVolcEngine(prompt, model =
'doubao-seedream-4-5-251128', size = '2048x2048', watermark = true) {
  const validSize = getValidSize(size);

  console.log('开始生成图像，参数:', { prompt, model, size:
validSize, watermark });

  try {
    const requestBody = {
      model,
      prompt,
      size: validSize,
      response_format: 'url',
      watermark
    };

    console.log('API 请求体:', JSON.stringify(requestBody, null, 2));

    const response = await fetch(VOLCENGINE_API_URL, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${VOLCENGINE_API_KEY}`
      },
      body: JSON.stringify(requestBody)
    });

    const responseText = await response.text();
    console.log('API 响应状态:', response.status);
    console.log('API 响应内容:', responseText);

    if (!response.ok) {
      let errorMessage = `API 请求失败 (${response.status})`;
      try {
        const errorData = JSON.parse(responseText);
        errorMessage = errorData.error?.message || errorData.message
|| errorMessage;
      } catch (e) {
        errorMessage = responseText || errorMessage;
      }
      throw new Error(errorMessage);
    }

    const data = JSON.parse(responseText);
```

```
    if (!data.data || !data.data[0] || !data.data[0].url) {
      throw new Error('API 返回数据格式异常: ' +
JSON.stringify(data));
    }

    console.log('图像生成成功，URL:', data.data[0].url);
    return data.data[0].url;

  } catch (error) {
    console.error('火山引擎 API 调用失败详情:', {
      error: error.message,
      stack: error.stack,
      prompt,
      model,
      size: validSize
    });
    throw error;
  }
}

export async function generateJiangsuCultureImage(culturalElement,
style = '中国风', size = '2048x2048') {
  const stylePrompts = {
    '中国风': '中国传统绘画风格，水墨意境，古典韵味',
    '水墨风格': '水墨画风格，意境深远，笔墨淋漓',
    '现代简约': '现代简约设计风格，简洁大方，时尚前卫',
    '复古风格': '复古怀旧风格，年代感强，经典韵味',
    '未来科技': '未来科技感风格，赛博朋克，霓虹光影',
    '卡通可爱': '卡通可爱风格，色彩明亮，童趣十足',
    '油画风格': '油画艺术风格，笔触厚重，色彩浓郁',
    '扁平化': '扁平化设计风格，简洁图形，现代感强',
    '传统风格': '传统文化风格，古典元素，民族特色'
  };

  const styleDesc = stylePrompts[style] || stylePrompts['中国风'];
  const prompt = `${culturalElement}, ${styleDesc}, 高清画质，精细细
节，艺术作品，专业摄影`;

  console.log('生成江苏文化图像:', { culturalElement, style,
prompt });

  return await generateImageWithVolcEngine(prompt, 'doubao-seedream-
4-5-251128', size);
```

```
}

/**
 * 生成文化设计的文本描述
 * @param {string} culturalElement - 文化元素
 * @param {string} style - 设计风格
 * @param {string} purpose - 设计用途
 * @returns {Promise<string>} - 生成的文本描述
 */
export async function generateCultureDesign(culturalElement, style = '中国风', purpose = '海报') {
  // 由于火山引擎API主要用于图像生成，这里创建一个模拟函数
  // 返回预设的文化设计文本描述
  const descriptions = {
    '海报': `江苏${culturalElement}主题海报设计，采用${style}风格，融合传统元素与现代设计理念。画面中心展示${culturalElement}的核心元素，周围配以江苏特色的装饰图案，色彩以红色和金色为主，体现传统文化的喜庆与庄重。整体布局平衡，视觉冲击力强，适合作为文化宣传用途。`,
    '标志': `江苏${culturalElement}主题标志设计，采用${style}风格，简洁明快，易于识别。标志以${culturalElement}的典型元素为基础，进行抽象化处理，形成独特的视觉符号。色彩搭配和谐，既体现传统文化特色，又具有现代感。适合作为文化品牌的视觉标识。`,
    '插画': `江苏${culturalElement}主题插画设计，采用${style}风格，细腻生动，富有故事性。插画描绘了${culturalElement}的典型场景，人物形象饱满，场景细节丰富，色彩搭配协调，营造出浓厚的文化氛围。适合作为文化书籍、杂志的插图使用。`,
    '包装': `江苏${culturalElement}主题包装设计，采用${style}风格，美观实用，富有文化内涵。包装以${culturalElement}的元素为装饰，色彩搭配和谐，材质选择考究，既保护产品，又能传达文化信息。适合作为文化产品的包装使用。`
  };

  return descriptions[purpose] || `江苏${culturalElement}主题${purpose}设计，采用${style}风格，融合传统与现代元素，体现江苏文化的独特魅力。`;
}

// ========== FILE: ai-culture-design\src\App.vue ==========
<script setup>
import { ref, onMounted, onUnmounted } from 'vue'
import { useRouter } from 'vue-router'
import { ElMessage } from 'element-plus'
import { removeToken } from '@/utils/api'
import '@surely-vue/table/dist/index.less';
```

```
const router = useRouter()
const footerRef = ref(null)

const menu = [
  {
    title: '首页',
    to: '/',
    children: []
  },
  {
    title: '江苏非遗',
    to: '/feiyi',
    children: [
      { title: '国家级项目', to: '/feiyi' },
      { title: '省级项目', to: '/feiyi' },
      { title: '项目分类', to: '/feiyi' }
    ]
  },
  {
    title: '江苏文化库',
    to: '/culture',
    children: [
      { title: '历史名人', to: '/celebrities' },
      { title: '传统艺术', to: '/traditional-arts' }
    ]
  },
  {
    title: 'AI 设计',
    to: '/design',
    children: []
  },
  {
    title: '江苏图库',
    to: '/gallery',
    children: []
  },
  {
    title: '我的作品',
    to: '/works',
    children: []
  }
]
```

```
const isLogin = ref(false)
const username = ref('')
const avatar =
ref('https://api.dicebear.com/9.x/avataaars/svg?seed=default')  // 默
认头像
const isScrolled = ref(false)

function handleScroll() {
  isScrolled.value = window.scrollY > 10
}

function updateUserInfo() {
  isLogin.value = localStorage.getItem('isLogin') === 'true';
  username.value = localStorage.getItem('username') || '';
  avatar.value = localStorage.getItem('avatar') ||
'https://api.dicebear.com/9.x/avataaars/svg?seed=default';
}

// 波浪动画效果
function handleMouseMove(event) {
  if (!footerRef.value) return

  const footer = footerRef.value
  const rect = footer.getBoundingClientRect()

  // 检查鼠标是否在页脚区域内
  if (event.clientX < rect.left || event.clientX > rect.right ||
      event.clientY < rect.top || event.clientY > rect.bottom) {
    resetWaves()
    return
  }

  const x = event.clientX - rect.left
  const y = event.clientY - rect.top
  const centerX = rect.width / 2
  const centerY = rect.height / 2

  // 计算鼠标与中心的距离比例
  const ratioX = (x - centerX) / centerX
  const ratioY = (y - centerY) / centerY

  // 获取波浪元素
  const waves = footer.querySelectorAll('.wave')
  waves.forEach((wave, index) => {
```

```
    // 不同波浪的振幅和延迟
    const amplitude = 10 + index * 5
    const delay = index * 0.1

    // 计算波浪的位移
    const offsetX = ratioX * amplitude
    const offsetY = ratioY * amplitude

    // 应用变换
    wave.style.transform = `translate(${offsetX}px, ${offsetY}px)
rotate(${ratioX * 5}deg)`
    wave.style.opacity = 0.3 + Math.abs(ratioX) * 0.5
  })
}

// 重置波浪位置
function resetWaves() {
  if (!footerRef.value) return

  const waves = footerRef.value.querySelectorAll('.wave')
  waves.forEach(wave => {
    wave.style.transform = 'translate(0, 0) rotate(0deg)'
    wave.style.opacity = 0.5
  })
}

onMounted(() => {
  updateUserInfo()
  router.afterEach(() => updateUserInfo())
  window.addEventListener('scroll', handleScroll)
  document.addEventListener('mousemove', handleMouseMove)
  document.addEventListener('mouseleave', resetWaves)
})

onUnmounted(() => {
  window.removeEventListener('scroll', handleScroll)
  document.removeEventListener('mousemove', handleMouseMove)
  document.removeEventListener('mouseleave', resetWaves)
})



function goLogin() {
  router.push('/login')
}
```

```
function logout() {
  removeToken()
  localStorage.removeItem('isLogin')
  localStorage.removeItem('username')
  localStorage.removeItem('nickname')
  localStorage.removeItem('avatar')
  isLogin.value = false
  ElMessage.success('已退出登录')
  router.push('/login')
}

function goProfile() {
  router.push('/profile')
}
</script>

<template>
  <!-- 导航栏 -->
  <nav :class="['navbar', { 'navbar-scrolled': isScrolled }]">
    <div class="nav-menu">
      <div v-for="item in menu" :key="item.title" class="nav-item-
container">
        <router-link v-if="item.children.length === 0" :to="item.to"
class="nav-item" active-class="active">
          {{ item.title }}
        </router-link>
        <div v-else class="nav-item-with-dropdown">
          <router-link :to="item.to" class="nav-item" active-
class="active">
            {{ item.title }}
            <i class="el-icon-arrow-down el-icon--right"></i>
          </router-link>
          <div class="dropdown-menu">
            <router-link
              v-for="child in item.children"
              :key="child.title"
              :to="child.to"
              class="dropdown-item"
            >
              {{ child.title }}
            </router-link>
          </div>
        </div>
```

```
      </div>
    </div>

    <div v-if="!isLogin" class="login-area">
      <button class="login-btn" @click="goLogin">
        <svg width="18" height="18" viewBox="0 0 24 24" fill="none">
          <path d="M20 21V19C20 17.9391 19.5786 16.9217 18.8284
16.1716C18.0783 15.4214 17.0609 15 16 15H8C6.93913 15 5.92172 15.4214
5.17157 16.1716C4.42143 16.9217 4 17.9391 4 19V21"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"/>
          <circle cx="12" cy="7" r="4" stroke="currentColor" stroke-
width="2" stroke-linecap="round" stroke-linejoin="round"/>
        </svg>
        <span>登录</span>
      </button>
    </div>

    <div v-else class="user-area">
      <el-dropdown trigger="click">
        <span class="user-dropdown">
          <div class="user-avatar">
            <el-avatar :src="avatar" />
            <span class="online-status"></span>
          </div>
          <span class="user-name">{{ username }}</span>
          <svg class="dropdown-arrow" width="16" height="16"
viewBox="0 0 24 24" fill="none">
            <path d="M6 9L12 15L18 9" stroke="currentColor" stroke-
width="2" stroke-linecap="round" stroke-linejoin="round"/>
          </svg>
        </span>
        <template #dropdown>
          <el-dropdown-menu>
            <el-dropdown-item @click="goProfile">
              <svg width="16" height="16" viewBox="0 0 24 24"
fill="none">
                <path d="M20 21V19C20 17.9391 19.5786 16.9217 18.8284
16.1716C18.0783 15.4214 17.0609 15 16 15H8C6.93913 15 5.92172 15.4214
5.17157 16.1716C4.42143 16.9217 4 17.9391 4 19V21"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"/>
                <circle cx="12" cy="7" r="4" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/>
```

```
              </svg>
              个人主页
            </el-dropdown-item>
            <el-dropdown-item divided @click="logout">
              <svg width="16" height="16" viewBox="0 0 24 24"
fill="none">
                <path d="M9 21H5C4.46957 21 3.96086 20.7893 3.58579
20.4142C3.21071 20.0391 3 19.5304 3 19V5C3 4.46957 3.21071 3.96086
3.58579 3.58579C3.96086 3.21071 4.46957 3 5 3H9"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"/>
                <path d="M16 17L21 12L16 7" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"/>
                <path d="M21 12H9" stroke="currentColor" stroke-
width="2" stroke-linecap="round" stroke-linejoin="round"/>
              </svg>
              退出登录
            </el-dropdown-item>
          </el-dropdown-menu>
        </template>
      </el-dropdown>
    </div>
  </nav>
  <!-- 页面切换过渡效果 -->
  <router-view v-slot="{ Component }">
    <transition name="page-transition" mode="out-in">
      <component :is="Component" />
    </transition>
  </router-view>

  <!-- 全局页脚 -->
  <footer class="page-footer" ref="footerRef">
    <div class="wave-container">
      <div v-for="i in 3" :key="i" :class="`wave wave-${i}`"></div>
    </div>
    <div class="footer-content">
      <p class="copyright">© 2024 文化传承与创新平台 版权所有</p>
      <p class="contact">联系电话：010-12345678 | 邮箱：
contact@culture.com</p>
      <p class="icp">京 ICP 备 12345678 号</p>
    </div>
  </footer>
</template>
```

```
<style scoped>
/* 导航栏 - 透明导航栏设计 */
.navbar {
  background-color: transparent;
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
  padding: 20px 30px;
  margin: 0;
  width: 100%;
  display: flex;
  align-items: center;
  box-shadow: none;
  z-index: 1000;
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  transition: all 0.4s cubic-bezier(0.25, 0.46, 0.45, 0.94); /* 更平滑的缓动函数 */
}

/* 滚动时的样式变化 */
.navbar-scrolled {
  background-color: white;
  backdrop-filter: none;
  -webkit-backdrop-filter: none;
  padding: 16px 30px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);
}

.nav-menu {
  display: flex;
  flex: 1;
  justify-content: flex-start;
  gap: 0;
}

.nav-item-container {
  position: relative;
}

.nav-item {
  color: white;
  font-family: '微软雅黑', Arial, sans-serif;
```

```
  font-weight: 400;
  font-size: 15px;
  cursor: pointer;
  padding: 10px 18px;
  transition: all 0.3s cubic-bezier(0.25, 0.46, 0.45, 0.94); /* 更平滑的缓动函数 */
  display: inline-block;
  text-decoration: none;
  border-radius: 2px;
  position: relative;
  text-shadow: 0 1px 2px rgba(0, 0, 0, 0.5); /* 添加文字阴影，增强可读性 */
}

.nav-item:hover {
  color: #D32F2F;
  background-color: rgba(255, 255, 255, 0.1);
  text-shadow: none;
}

.nav-item::after {
  content: '';
  position: absolute;
  bottom: 0;
  left: 50%;
  width: 0;
  height: 2px;
  background-color: #D32F2F;
  transition: all 0.3s cubic-bezier(0.25, 0.46, 0.45, 0.94);
  transform: translateX(-50%);
}

.nav-item:hover::after {
  width: 80%;
}

.router-link-active {
  color: #D32F2F !important;
  font-weight: 500;
  text-shadow: none;
}

.router-link-active::after {
  width: 80%;
```

```
}

/* 滚动时的导航项样式 */
.navbar-scrolled .nav-item {
  color: #333333;
  text-shadow: none;
}

.navbar-scrolled .nav-item:hover {
  background-color: rgba(211, 47, 47, 0.05);
}

.navbar-scrolled .router-link-active {
  background-color: rgba(211, 47, 47, 0.05);
}

/* 下拉菜单样式 - 参考中国非物质文化遗产网风格 */
.nav-item-with-dropdown {
  position: relative;
}

.dropdown-menu {
  position: absolute;
  top: 100%;
  left: 0;
  min-width: 170px;
  background-color: white;
  border: 1px solid #E8E8E8;
  box-shadow: 0 3px 12px rgba(0, 0, 0, 0.1);
  padding: 6px 0;
  margin-top: 4px;
  opacity: 0;
  visibility: hidden;
  transform: translateY(-10px);
  transition: all 0.3s cubic-bezier(0.25, 0.46, 0.45, 0.94); /* 更平
滑的缓动函数 */
  z-index: 1001;
  border-radius: 4px;
}

.nav-item-with-dropdown:hover .dropdown-menu {
  opacity: 1;
  visibility: visible;
  transform: translateY(0);
```

```
}

.dropdown-item {
  display: block;
  padding: 8px 16px;
  color: #333333;
  font-family: '微软雅黑', Arial, sans-serif;
  font-size: 14px;
  transition: all 0.3s cubic-bezier(0.25, 0.46, 0.45, 0.94); /* 更平
滑的缓动函数 */
  white-space: nowrap;
  text-decoration: none;
  border-radius: 2px;
  margin: 0 4px;
}

.dropdown-item:hover {
  color: #D32F2F;
  background-color: rgba(211, 47, 47, 0.05);
}

/* 登录和用户区域 */
.login-area {
  margin-left: auto;
}

/* 页面切换过渡效果 */
.page-transition-enter-active,
.page-transition-leave-active {
  transition: all 0.6s cubic-bezier(0.25, 0.46, 0.45, 0.94);
}

.page-transition-enter-from {
  opacity: 0;
  transform: translateY(20px);
}

.page-transition-leave-to {
  opacity: 0;
  transform: translateY(-20px);
}

.login-area .el-button {
  font-size: 16px;
```

```
  padding: 10px 20px;
}

/* 登录按钮 - 高级感颜色方案设计 */
/*
 * 色彩体系：
 * - 主色：深邃中国红 #8B1538 (传承红)
 * - 辅助色：奢华金 #C9A962 (点缀金)
 * - 高光：珍珠白渐变
 * - 阴影：深红到紫红过渡
 */
.login-btn {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  padding: 12px 28px;
  background: linear-gradient(145deg,
    #A01D3E 0%,
    #8B1538 35%,
    #6D0F2C 65%,
    #8B1538 100%);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  color: #FEFEFE;
  border: 1px solid rgba(201, 169, 98, 0.35);
  border-radius: 30px;
  font-size: 15px;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.4s cubic-bezier(0.25, 0.46, 0.45, 0.94);
  box-shadow:
    0 4px 24px rgba(139, 21, 56, 0.35),
    0 2px 8px rgba(109, 15, 44, 0.25),
    0 0 0 1px rgba(201, 169, 98, 0.15) inset,
    0 1px 0 rgba(255, 255, 255, 0.15) inset;
  position: relative;
  overflow: hidden;
  letter-spacing: 0.5px;
  text-shadow: 0 1px 2px rgba(109, 15, 44, 0.5);
}

.login-btn::before {
  content: '';
  position: absolute;
```

```
  top: 0;
  left: -100%;
  width: 100%;
  height: 100%;
  background: linear-gradient(90deg,
    transparent 0%,
    rgba(201, 169, 98, 0.3) 25%,
    rgba(255, 255, 255, 0.4) 50%,
    rgba(201, 169, 98, 0.3) 75%,
    transparent 100%);
  transition: left 0.7s cubic-bezier(0.25, 0.46, 0.45, 0.94);
}

.login-btn::after {
  content: '';
  position: absolute;
  top: -50%;
  left: -50%;
  right: -50%;
  bottom: -50%;
  background: radial-gradient(ellipse at 30% 30%,
    rgba(201, 169, 98, 0.15) 0%,
    rgba(201, 169, 98, 0.08) 30%,
    transparent 60%);
  opacity: 0;
  transition: opacity 0.5s ease;
  pointer-events: none;
}

/* 悬停状态 - 提升亮度与金色光晕 */
.login-btn:hover {
  transform: translateY(-3px) scale(1.02);
  background: linear-gradient(145deg,
    #B8234A 0%,
    #A01D3E 35%,
    #8B1538 65%,
    #A01D3E 100%);
  box-shadow:
    0 8px 36px rgba(139, 21, 56, 0.45),
    0 4px 16px rgba(109, 15, 44, 0.3),
    0 0 0 1px rgba(201, 169, 98, 0.4) inset,
    0 1px 0 rgba(255, 255, 255, 0.25) inset,
    0 0 30px rgba(201, 169, 98, 0.15);
  border-color: rgba(201, 169, 98, 0.6);
```

```
  color: #FFFFFF;
}

.login-btn:hover::before {
  left: 100%;
}

.login-btn:hover::after {
  opacity: 1;
}

/* 激活/按下状态 - 深色收缩反馈 */
.login-btn:active {
  transform: translateY(-1px) scale(0.98);
  background: linear-gradient(145deg,
    #6D0F2C 0%,
    #5A0C24 35%,
    #4A0A1E 65%,
    #5A0C24 100%);
  box-shadow:
    0 2px 12px rgba(109, 15, 44, 0.4),
    0 1px 4px rgba(74, 10, 30, 0.3),
    0 0 0 1px rgba(201, 169, 98, 0.2) inset;
  border-color: rgba(201, 169, 98, 0.4);
}

/* 禁用状态 */
.login-btn:disabled {
  background: linear-gradient(145deg,
    #9A9A9A 0%,
    #7A7A7A 50%,
    #8A8A8A 100%);
  border-color: rgba(180, 180, 180, 0.3);
  box-shadow:
    0 2px 8px rgba(0, 0, 0, 0.1),
    0 0 0 1px rgba(255, 255, 255, 0.1) inset;
  cursor: not-allowed;
  opacity: 0.7;
  transform: none;
}

.login-btn:disabled:hover {
  transform: none;
  box-shadow:
```

```
    0 2px 8px rgba(0, 0, 0, 0.1),
    0 0 0 1px rgba(255, 255, 255, 0.1) inset;
}

.login-btn svg {
  transition: all 0.3s cubic-bezier(0.25, 0.46, 0.45, 0.94);
  filter: drop-shadow(0 1px 2px rgba(74, 10, 30, 0.4));
  color: #FEFEFE;
}

.login-btn:hover svg {
  transform: translateX(2px);
  filter: drop-shadow(0 2px 4px rgba(201, 169, 98, 0.3));
}

.login-btn span {
  position: relative;
  z-index: 1;
}

/* 滚动时的登录按钮样式 - 白色背景适配 */
.navbar-scrolled .login-btn {
  background: linear-gradient(145deg,
    #A01D3E 0%,
    #8B1538 35%,
    #6D0F2C 65%,
    #8B1538 100%);
  border-color: rgba(139, 21, 56, 0.4);
  box-shadow:
    0 4px 20px rgba(139, 21, 56, 0.25),
    0 2px 8px rgba(109, 15, 44, 0.15),
    0 0 0 1px rgba(201, 169, 98, 0.2) inset;
}

.navbar-scrolled .login-btn:hover {
  box-shadow:
    0 8px 32px rgba(139, 21, 56, 0.35),
    0 4px 12px rgba(109, 15, 44, 0.2),
    0 0 0 1px rgba(201, 169, 98, 0.5) inset,
    0 0 24px rgba(201, 169, 98, 0.12);
  border-color: rgba(201, 169, 98, 0.7);
}

/* 焦点状态 - 无障碍设计 */
```

```
.login-btn:focus-visible {
  outline: none;
  box-shadow:
    0 4px 24px rgba(139, 21, 56, 0.35),
    0 2px 8px rgba(109, 15, 44, 0.25),
    0 0 0 3px rgba(201, 169, 98, 0.5),
    0 0 0 1px rgba(201, 169, 98, 0.15) inset;
}

/* 用户区域 */
.user-area {
  margin-left: auto;
  display: flex;
  align-items: center;
}

/* 用户下拉链接 */
.user-dropdown {
  cursor: pointer;
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 6px 12px 6px 6px;
  background: rgba(255, 255, 255, 0.1);
  backdrop-filter: blur(10px);
  border: 1px solid rgba(255, 255, 255, 0.2);
  border-radius: 30px;
  transition: all 0.3s ease;
}

.navbar-scrolled .user-dropdown {
  background: rgba(0, 0, 0, 0.05);
  border-color: rgba(0, 0, 0, 0.1);
}

.user-dropdown:hover {
  background: rgba(255, 255, 255, 0.2);
  border-color: rgba(255, 255, 255, 0.3);
  transform: translateY(-2px);
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}

.navbar-scrolled .user-dropdown:hover {
  background: rgba(0, 0, 0, 0.08);
```

```
  border-color: rgba(192, 12, 0, 0.3);
}

/* 用户头像容器 */
.user-avatar {
  position: relative;
}

.el-avatar {
  width: 36px;
  height: 36px;
  border: 2px solid rgba(255, 255, 255, 0.5);
  transition: all 0.3s ease;
}

.navbar-scrolled .el-avatar {
  border-color: rgba(192, 12, 0, 0.3);
}

.user-dropdown:hover .el-avatar {
  border-color: #C00C00;
}

/* 在线状态指示器 */
.online-status {
  position: absolute;
  bottom: 0;
  right: 0;
  width: 10px;
  height: 10px;
  background: #22c55e;
  border: 2px solid white;
  border-radius: 50%;
  animation: pulse 2s ease-in-out infinite;
}

@keyframes pulse {
  0%, 100% {
    transform: scale(1);
    opacity: 1;
  }
  50% {
    transform: scale(1.2);
    opacity: 0.7;
```

```
  }
}

/* 用户名 */
.user-name {
  font-weight: 500;
  color: white;
  font-size: 15px;
  text-shadow: 0 1px 2px rgba(0, 0, 0, 0.3);
  max-width: 100px;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.navbar-scrolled .user-name {
  color: #333;
  text-shadow: none;
}

/* 下拉箭头 */
.dropdown-arrow {
  color: rgba(255, 255, 255, 0.8);
  transition: transform 0.3s ease;
}

.navbar-scrolled .dropdown-arrow {
  color: #666;
}

.user-dropdown:hover .dropdown-arrow {
  transform: rotate(180deg);
  color: #C00C00;
}

/* 下拉菜单样式 */
:deep(.el-dropdown-menu) {
  padding: 8px;
  border-radius: 12px;
  border: 1px solid rgba(0, 0, 0, 0.08);
  box-shadow: 0 10px 40px rgba(0, 0, 0, 0.12);
  min-width: 160px;
}
```

```
:deep(.el-dropdown-menu__item) {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 12px 16px;
  border-radius: 8px;
  font-size: 14px;
  color: #333;
  transition: all 0.3s ease;
}

:deep(.el-dropdown-menu__item svg) {
  color: #666;
  flex-shrink: 0;
}

:deep(.el-dropdown-menu__item:hover) {
  background: linear-gradient(135deg, rgba(192, 12, 0, 0.08) 0%,
rgba(192, 12, 0, 0.05) 100%);
  color: #C00C00;
}

:deep(.el-dropdown-menu__item:hover svg) {
  color: #C00C00;
}

:deep(.el-dropdown-menu__item--divided) {
  margin-top: 8px;
  border-top: 1px solid #f0f0f0;
  padding-top: 16px;
}

/* 滚动时的用户区域样式 - 保留兼容性 */
.navbar-scrolled .el-dropdown-link {
  color: #333333;
  text-shadow: none;
}

/* 响应式设计 */
@media (max-width: 1199px) {
  .nav-item {
    font-size: 13px;
    padding: 6px 12px;
  }
```

```
}

@media (max-width: 767px) {
  .navbar {
    padding: 10px 15px;
  }

  .nav-menu {
    flex-wrap: wrap;
  }

  .nav-item {
    font-size: 12px;
    padding: 4px 8px;
    margin-right: 8px;
  }

  .login-btn {
    padding: 8px 16px;
    font-size: 13px;
  }

  .login-btn svg {
    width: 16px;
    height: 16px;
  }

  .user-dropdown {
    padding: 4px 10px 4px 4px;
  }

  .el-avatar {
    width: 32px;
    height: 32px;
  }

  .user-name {
    font-size: 13px;
    max-width: 60px;
  }

  .online-status {
    width: 8px;
    height: 8px;
```

```
  }

  .dropdown-arrow {
    width: 14px;
    height: 14px;
  }
}

/* 页脚样式 */
.page-footer {
  background-color: #333333;
  color: white;
  padding: 40px 0;
  margin-top: 80px;
  position: relative;
        if (StrUtil.isNotEmpty(pageDTO.getKeyword())) {
            wrapper.and(w -> w.like(Heritage::getName,
pageDTO.getKeyword())
                    .or().like(Heritage::getDescription,
pageDTO.getKeyword())
                    .or().like(Heritage::getTags,
pageDTO.getKeyword()));
        }

        wrapper.orderByDesc(Heritage::getViewCount);
        wrapper.orderByDesc(Heritage::getCreateTime);

        Page<Heritage> heritagePage = heritageMapper.selectPage(page,
wrapper);

        Page<HeritageVO> voPage = new
Page<>(heritagePage.getCurrent(), heritagePage.getSize(),
heritagePage.getTotal());
        voPage.setRecords(heritagePage.getRecords().stream()
                .map(this::convertToVO)
                .collect(Collectors.toList()));

        return voPage;
    }

    @Override
    @Cacheable(value = "heritage:detail", key = "#id")
    public HeritageVO getDetailById(Long id) {
        Heritage heritage = heritageMapper.selectById(id);
```

```
        if (heritage == null) {
            return null;
        }
        return convertToVO(heritage);
    }

    @Override
    @CacheEvict(value = "heritage:detail", key = "#id")
    public void incrementViewCount(Long id) {
        heritageMapper.update(null,
                new LambdaUpdateWrapper<Heritage>()
                        .setSql("view_count = view_count + 1")
                        .eq(Heritage::getId, id)
        );
    }

    @Override
    @CacheEvict(value = "heritage:detail", key = "#id")
    public void incrementLikeCount(Long id) {
        heritageMapper.update(null,
                new LambdaUpdateWrapper<Heritage>()
                        .setSql("like_count = like_count + 1")
                        .eq(Heritage::getId, id)
        );
    }

    private HeritageVO convertToVO(Heritage heritage) {
        HeritageVO vo = BeanUtil.copyProperties(heritage,
HeritageVO.class);

        if (StrUtil.isNotEmpty(heritage.getImages())) {
            vo.setImages(heritage.getImages().split(","));
        }
        if (StrUtil.isNotEmpty(heritage.getTags())) {
            vo.setTags(heritage.getTags().split(","));
        }

        vo.setLevelName(getLevelName(heritage.getLevel()));

        return vo;
    }

    private String getLevelName(Integer level) {
        if (level == null) return "未知";
```

```
        return switch (level) {
            case 1 -> "国家级";
            case 2 -> "省级";
            case 3 -> "市级";
            case 4 -> "县级";
            default -> "未知";
        };
    }
}

// ========== FILE:
backend\src\main\java\com\cultural\service\impl\LikeServiceImpl.java
==========
package com.cultural.service.impl;

import
com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.cultural.entity.LikeRecord;
import com.cultural.mapper.LikeRecordMapper;
import com.cultural.service.LikeService;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class LikeServiceImpl implements LikeService {

    private final LikeRecordMapper likeRecordMapper;

    @Override
    public boolean toggleLike(Long userId, String targetType, Long
targetId) {
        if (userId == null) {
            return false;
        }

        LambdaQueryWrapper<LikeRecord> wrapper = new
LambdaQueryWrapper<>();
        wrapper.eq(LikeRecord::getUserId, userId)
                .eq(LikeRecord::getTargetType, targetType)
                .eq(LikeRecord::getTargetId, targetId);

        LikeRecord existing = likeRecordMapper.selectOne(wrapper);
```

```
        if (existing != null) {
            int newStatus = existing.getStatus() == 1 ? 0 : 1;
            existing.setStatus(newStatus);
            likeRecordMapper.updateById(existing);
            return newStatus == 1;
        } else {
            LikeRecord likeRecord = new LikeRecord();
            likeRecord.setUserId(userId);
            likeRecord.setTargetType(targetType);
            likeRecord.setTargetId(targetId);
            likeRecord.setStatus(1);
            likeRecordMapper.insert(likeRecord);
            return true;
        }
    }

    @Override
    public boolean isLiked(Long userId, String targetType, Long
targetId) {
        if (userId == null) {
            return false;
        }
        Long count = likeRecordMapper.selectCount(
                new LambdaQueryWrapper<LikeRecord>()
                        .eq(LikeRecord::getUserId, userId)
                        .eq(LikeRecord::getTargetType, targetType)
                        .eq(LikeRecord::getTargetId, targetId)
                        .eq(LikeRecord::getStatus, 1)
        );
        return count != null && count > 0;
    }
}

// ========== FILE:
backend\src\main\java\com\cultural\service\impl\NewsServiceImpl.java
==========
package com.cultural.service.impl;

import cn.hutool.core.bean.BeanUtil;
import cn.hutool.core.util.StrUtil;
import
com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import
com.baomidou.mybatisplus.core.conditions.update.LambdaUpdateWrapper;
```

```
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.cultural.dto.PageDTO;
import com.cultural.entity.News;
import com.cultural.mapper.NewsMapper;
import com.cultural.service.NewsService;
import com.cultural.vo.NewsVO;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;

@Slf4j
@Service
@RequiredArgsConstructor
public class NewsServiceImpl implements NewsService {

    private final NewsMapper newsMapper;

    @Override
    @Cacheable(value = "news:list", key = "#pageDTO.pageNum + ':' +
#pageDTO.pageSize + ':' + #pageDTO.category")
    public Page<NewsVO> getPageList(PageDTO pageDTO) {
        Page<News> page = new Page<>(pageDTO.getPageNum(),
pageDTO.getPageSize());

        LambdaQueryWrapper<News> wrapper = new
LambdaQueryWrapper<>();
        wrapper.eq(News::getStatus, 1);

        if (StrUtil.isNotEmpty(pageDTO.getCategory())) {
            wrapper.eq(News::getCategory, pageDTO.getCategory());
        }
        if (StrUtil.isNotEmpty(pageDTO.getKeyword())) {
            wrapper.like(News::getTitle, pageDTO.getKeyword())
                    .or()
                    .like(News::getSummary, pageDTO.getKeyword());
        }

        wrapper.orderByDesc(News::getPublishTime);

        Page<News> newsPage = newsMapper.selectPage(page, wrapper);
```

```
        Page<NewsVO> voPage = new Page<>(newsPage.getCurrent(),
newsPage.getSize(), newsPage.getTotal());
        voPage.setRecords(newsPage.getRecords().stream()
                .map(this::convertToVO)
                .toList());

        return voPage;
    }

    @Override
    @Cacheable(value = "news:detail", key = "#id")
    public NewsVO getDetailById(Long id) {
        News news = newsMapper.selectById(id);
        if (news == null) {
            return null;
        }
        return convertToVO(news);
    }

    @Override
    @CacheEvict(value = "news:detail", key = "#id")
    public void incrementViewCount(Long id) {
        newsMapper.update(null,
                new LambdaUpdateWrapper<News>()
                        .setSql("view_count = view_count + 1")
                        .eq(News::getId, id)
        );
    }

    private NewsVO convertToVO(News news) {
        return BeanUtil.copyProperties(news, NewsVO.class);
    }
}

// ========== FILE:
backend\src\main\java\com\cultural\service\impl\SearchServiceImpl.jav
a ==========
package com.cultural.service.impl;

import cn.hutool.core.bean.BeanUtil;
import cn.hutool.core.util.StrUtil;
import
com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
```

```
import com.cultural.dto.SearchDTO;
import com.cultural.entity.*;
import com.cultural.mapper.*;
import com.cultural.service.SearchService;
import com.cultural.vo.SearchResultVO;
import lombok.RequiredArgsConstructor;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class SearchServiceImpl implements SearchService {

    private final SearchHistoryMapper searchHistoryMapper;
    private final HeritageMapper heritageMapper;
    private final TraditionalArtMapper traditionalArtMapper;
    private final CelebrityMapper celebrityMapper;
    private final NewsMapper newsMapper;
    private final UserWorkMapper userWorkMapper;

    @Override
    public Page<SearchResultVO> search(SearchDTO searchDTO, Long
userId) {
        String keyword = searchDTO.getKeyword();
        if (StrUtil.isEmpty(keyword)) {
            return new Page<>();
        }

        saveSearchHistory(keyword, searchDTO.getSearchType(),
userId);

        Page<SearchResultVO> resultPage = new
Page<>(searchDTO.getPageNum(), searchDTO.getPageSize());
        List<SearchResultVO> results = new ArrayList<>();

        String searchType = searchDTO.getSearchType();

        if ("all".equals(searchType) ||
"heritage".equals(searchType)) {
            results.addAll(searchHeritage(keyword));
```

```
        }
        if ("all".equals(searchType) || "art".equals(searchType)) {
            results.addAll(searchArt(keyword));
        }
        if ("all".equals(searchType) ||
"celebrity".equals(searchType)) {
            results.addAll(searchCelebrity(keyword));
        }
        if ("all".equals(searchType) || "news".equals(searchType)) {
            results.addAll(searchNews(keyword));
        }
        if ("all".equals(searchType) || "work".equals(searchType)) {
            results.addAll(searchWork(keyword));
        }

        results.sort((a, b) ->
b.getViewCount().compareTo(a.getViewCount()));

        int start = (int) ((searchDTO.getPageNum() - 1) *
searchDTO.getPageSize());
        int end = Math.min(start + searchDTO.getPageSize(),
results.size());

        resultPage.setRecords(results.subList(start, end));
        resultPage.setTotal(results.size());

        return resultPage;
    }

    @Override
    public List<String> getHotKeywords(int limit) {
        List<SearchHistory> histories =
searchHistoryMapper.selectList(
                new LambdaQueryWrapper<SearchHistory>()
                        .select(SearchHistory::getKeyword)
                        .groupBy(SearchHistory::getKeyword)
                        .orderByDesc(SearchHistory::getCreateTime)
                        .last("LIMIT " + limit)
        );
        return histories.stream()
                .map(SearchHistory::getKeyword)
                .collect(Collectors.toList());
    }
```

```
    @Override
    public List<String> getUserSearchHistory(Long userId, int limit)
{
        if (userId == null) {
            return new ArrayList<>();
        }
        List<SearchHistory> histories =
searchHistoryMapper.selectList(
                new LambdaQueryWrapper<SearchHistory>()
                        .eq(SearchHistory::getUserId, userId)
                        .orderByDesc(SearchHistory::getCreateTime)
                        .last("LIMIT " + limit)
        );
        return histories.stream()
                .map(SearchHistory::getKeyword)
                .distinct()
                .collect(Collectors.toList());
    }

    @Override
    public void clearUserSearchHistory(Long userId) {
        if (userId != null) {
            searchHistoryMapper.delete(
                    new LambdaQueryWrapper<SearchHistory>()
                            .eq(SearchHistory::getUserId, userId)
            );
        }
    }

    @Async
    protected void saveSearchHistory(String keyword, String
searchType, Long userId) {
        SearchHistory history = new SearchHistory();
        history.setKeyword(keyword);
        history.setSearchType(searchType);
        history.setUserId(userId);
        searchHistoryMapper.insert(history);
    }

    private List<SearchResultVO> searchHeritage(String keyword) {
        List<Heritage> list = heritageMapper.selectList(
                new LambdaQueryWrapper<Heritage>()
                        .eq(Heritage::getStatus, 1)
                        .and(w -> w.like(Heritage::getName, keyword)
```

```
                                .or().like(Heritage::getDescription,
keyword)
                                .or().like(Heritage::getTags,
keyword))
                        .orderByDesc(Heritage::getViewCount)
                        .last("LIMIT 20")
        );
        return list.stream().map(h -> {
            SearchResultVO vo = new SearchResultVO();
            vo.setId(h.getId());
            vo.setType("heritage");
            vo.setTitle(h.getName());
            vo.setDescription(h.getDescription());
            vo.setCoverImage(h.getCoverImage());
            vo.setCategory(h.getCategory());
            vo.setCity(h.getCity());
            vo.setViewCount(h.getViewCount());
            vo.setLikeCount(h.getLikeCount());
            vo.setCreateTime(h.getCreateTime());
            return vo;
        }).collect(Collectors.toList());
    }

    private List<SearchResultVO> searchArt(String keyword) {
        List<TraditionalArt> list = traditionalArtMapper.selectList(
                new LambdaQueryWrapper<TraditionalArt>()
                        .eq(TraditionalArt::getStatus, 1)
                        .and(w -> w.like(TraditionalArt::getName,
keyword)
                                .or().like(TraditionalArt::getDescrip
tion, keyword)
                                .or().like(TraditionalArt::getTags,
keyword))
                        .orderByDesc(TraditionalArt::getViewCount)
                        .last("LIMIT 20")
        );
        return list.stream().map(a -> {
            SearchResultVO vo = new SearchResultVO();
            vo.setId(a.getId());
            vo.setType("art");
            vo.setTitle(a.getName());
            vo.setDescription(a.getDescription());
            vo.setCoverImage(a.getCoverImage());
            vo.setCategory(a.getCategory());
```

```
            vo.setCity(a.getCity());
            vo.setViewCount(a.getViewCount());
            vo.setLikeCount(a.getLikeCount());
            vo.setCreateTime(a.getCreateTime());
            return vo;
        }).collect(Collectors.toList());
    }

    private List<SearchResultVO> searchCelebrity(String keyword) {
        List<Celebrity> list = celebrityMapper.selectList(
                new LambdaQueryWrapper<Celebrity>()
                        .eq(Celebrity::getStatus, 1)
                        .and(w -> w.like(Celebrity::getName, keyword)
                                .or().like(Celebrity::getDescription,
keyword)
                                .or().like(Celebrity::getTags,
keyword))
                        .orderByDesc(Celebrity::getViewCount)
                        .last("LIMIT 20")
        );
        return list.stream().map(c -> {
            SearchResultVO vo = new SearchResultVO();
            vo.setId(c.getId());
            vo.setType("celebrity");
            vo.setTitle(c.getName());
            vo.setDescription(c.getDescription());
            vo.setCoverImage(c.getAvatar());
            vo.setCity(c.getCity());
            vo.setViewCount(c.getViewCount());
            vo.setLikeCount(c.getLikeCount());
            vo.setCreateTime(c.getCreateTime());
            return vo;
        }).collect(Collectors.toList());
    }

    private List<SearchResultVO> searchNews(String keyword) {
        List<News> list = newsMapper.selectList(
                new LambdaQueryWrapper<News>()
                        .eq(News::getStatus, 1)
                        .and(w -> w.like(News::getTitle, keyword)
                                .or().like(News::getSummary, keyword)
                                .or().like(News::getTags, keyword))
                        .orderByDesc(News::getViewCount)
                        .last("LIMIT 20")
```

```
        );
        return list.stream().map(n -> {
            SearchResultVO vo = new SearchResultVO();
            vo.setId(n.getId());
            vo.setType("news");
            vo.setTitle(n.getTitle());
            vo.setDescription(n.getSummary());
            vo.setCoverImage(n.getCoverImage());
            vo.setCategory(n.getCategory());
            vo.setViewCount(n.getViewCount());
            vo.setLikeCount(n.getLikeCount());
            vo.setCreateTime(n.getCreateTime());
            return vo;
        }).collect(Collectors.toList());
    }

    private List<SearchResultVO> searchWork(String keyword) {
        List<UserWork> list = userWorkMapper.selectList(
                new LambdaQueryWrapper<UserWork>()
                        .eq(UserWork::getStatus, 1)
                        .eq(UserWork::getIsPublic, 1)
                        .and(w -> w.like(UserWork::getTitle, keyword)
                                .or().like(UserWork::getDescription,
keyword)
                                .or().like(UserWork::getTags,
keyword))
                        .orderByDesc(UserWork::getViewCount)
                        .last("LIMIT 20")
        );
        return list.stream().map(w -> {
            SearchResultVO vo = new SearchResultVO();
            vo.setId(w.getId());
            vo.setType("work");
            vo.setTitle(w.getTitle());
            vo.setDescription(w.getDescription());
            vo.setCoverImage(w.getImageUrl());
            vo.setViewCount(w.getViewCount());
            vo.setLikeCount(w.getLikeCount());
            vo.setCreateTime(w.getCreateTime());
            return vo;
        }).collect(Collectors.toList());
    }
}
```

```
// ========== FILE:
backend\src\main\java\com\cultural\service\impl\TraditionalArtService
Impl.java ==========
package com.cultural.service.impl;

import cn.hutool.core.bean.BeanUtil;
import cn.hutool.core.util.StrUtil;
import
com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import
com.baomidou.mybatisplus.core.conditions.update.LambdaUpdateWrapper;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.cultural.dto.PageDTO;
import com.cultural.entity.TraditionalArt;
import com.cultural.mapper.TraditionalArtMapper;
import com.cultural.service.TraditionalArtService;
import com.cultural.vo.TraditionalArtVO;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;

@Slf4j
@Service
@RequiredArgsConstructor
public class TraditionalArtServiceImpl implements
TraditionalArtService {

    private final TraditionalArtMapper traditionalArtMapper;

    @Override
    @Cacheable(value = "arts:list", key = "#pageDTO.pageNum + ':' +
#pageDTO.pageSize + ':' + #pageDTO.category + ':' + #pageDTO.city")
    public Page<TraditionalArtVO> getPageList(PageDTO pageDTO) {
        Page<TraditionalArt> page = new Page<>(pageDTO.getPageNum(),
pageDTO.getPageSize());

        LambdaQueryWrapper<TraditionalArt> wrapper = new
LambdaQueryWrapper<>();
        wrapper.eq(TraditionalArt::getStatus, 1);

        if (StrUtil.isNotEmpty(pageDTO.getCategory())) {
```

```
            wrapper.eq(TraditionalArt::getCategory,
pageDTO.getCategory());
        }
        if (StrUtil.isNotEmpty(pageDTO.getCity())) {
            wrapper.eq(TraditionalArt::getCity, pageDTO.getCity());
        }
        if (StrUtil.isNotEmpty(pageDTO.getKeyword())) {
            wrapper.like(TraditionalArt::getName,
pageDTO.getKeyword())
                    .or()
                    .like(TraditionalArt::getDescription,
pageDTO.getKeyword());
        }

        wrapper.orderByDesc(TraditionalArt::getCreateTime);

        Page<TraditionalArt> artPage =
traditionalArtMapper.selectPage(page, wrapper);

        Page<TraditionalArtVO> voPage = new
Page<>(artPage.getCurrent(), artPage.getSize(), artPage.getTotal());
        voPage.setRecords(artPage.getRecords().stream()
                .map(this::convertToVO)
                .toList());

        return voPage;
    }

    @Override
    @Cacheable(value = "arts:detail", key = "#id")
    public TraditionalArtVO getDetailById(Long id) {
        TraditionalArt art = traditionalArtMapper.selectById(id);
        if (art == null) {
            return null;
        }
        return convertToVO(art);
    }

    @Override
    @CacheEvict(value = "arts:detail", key = "#id")
    public void incrementViewCount(Long id) {
        traditionalArtMapper.update(null,
                new LambdaUpdateWrapper<TraditionalArt>()
                        .setSql("view_count = view_count + 1")
```

```
                        .eq(TraditionalArt::getId, id)
        );
    }

    private TraditionalArtVO convertToVO(TraditionalArt art) {
        TraditionalArtVO vo = BeanUtil.copyProperties(art,
TraditionalArtVO.class);
        if (StrUtil.isNotEmpty(art.getImages())) {
            vo.setImages(art.getImages().split(","));
        }
        return vo;
    }
}

// ========== FILE:
backend\src\main\java\com\cultural\service\impl\UserWorkServiceImpl.j
ava ==========
package com.cultural.service.impl;

import cn.hutool.core.bean.BeanUtil;
import cn.hutool.core.util.StrUtil;
import
com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import
com.baomidou.mybatisplus.core.conditions.update.LambdaUpdateWrapper;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.cultural.common.exception.BusinessException;
import com.cultural.dto.PageDTO;
import com.cultural.dto.UserWorkDTO;
import com.cultural.entity.User;
import com.cultural.entity.UserWork;
import com.cultural.mapper.UserMapper;
import com.cultural.mapper.UserWorkMapper;
import com.cultural.security.LoginUser;
import com.cultural.service.UserWorkService;
import com.cultural.vo.UserWorkVO;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.security.core.Authentication;
import
org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

```
@Slf4j
@Service
@RequiredArgsConstructor
public class UserWorkServiceImpl implements UserWorkService {

    private final UserWorkMapper userWorkMapper;
    private final UserMapper userMapper;

    @Override
    public Page<UserWorkVO> getPageList(PageDTO pageDTO) {
        Page<UserWork> page = new Page<>(pageDTO.getPageNum(),
pageDTO.getPageSize());

        LambdaQueryWrapper<UserWork> wrapper = new
LambdaQueryWrapper<>();
        wrapper.eq(UserWork::getStatus, 1)
                .eq(UserWork::getIsPublic, 1);

        if (StrUtil.isNotEmpty(pageDTO.getKeyword())) {
            wrapper.like(UserWork::getTitle, pageDTO.getKeyword())
                    .or()
                    .like(UserWork::getDescription,
pageDTO.getKeyword());
        }
        if (StrUtil.isNotEmpty(pageDTO.getCulturalElement())) {
            wrapper.like(UserWork::getCulturalElement,
pageDTO.getCulturalElement());
        }

        wrapper.orderByDesc(UserWork::getCreateTime);

        Page<UserWork> workPage = userWorkMapper.selectPage(page,
wrapper);

        Page<UserWorkVO> voPage = new Page<>(workPage.getCurrent(),
workPage.getSize(), workPage.getTotal());
        voPage.setRecords(workPage.getRecords().stream()
                .map(this::convertToVO)
                .toList());

        return voPage;
    }

    @Override
```

```
    public Page<UserWorkVO> getMyWorks(PageDTO pageDTO, Long userId)
{
        Page<UserWork> page = new Page<>(pageDTO.getPageNum(),
pageDTO.getPageSize());

        LambdaQueryWrapper<UserWork> wrapper = new
LambdaQueryWrapper<>();
        wrapper.eq(UserWork::getUserId, userId)
                .eq(UserWork::getStatus, 1);

        if (StrUtil.isNotEmpty(pageDTO.getKeyword())) {
            wrapper.like(UserWork::getTitle, pageDTO.getKeyword());
        }

        wrapper.orderByDesc(UserWork::getCreateTime);

        Page<UserWork> workPage = userWorkMapper.selectPage(page,
wrapper);

        Page<UserWorkVO> voPage = new Page<>(workPage.getCurrent(),
workPage.getSize(), workPage.getTotal());
        voPage.setRecords(workPage.getRecords().stream()
                .map(this::convertToVO)
                .toList());

        return voPage;
    }

    @Override
    public UserWorkVO getDetailById(Long id) {
        UserWork work = userWorkMapper.selectById(id);
        if (work == null) {
            return null;
        }
        return convertToVO(work);
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public Long createWork(UserWorkDTO userWorkDTO, Long userId) {
        UserWork work = BeanUtil.copyProperties(userWorkDTO,
UserWork.class);
        work.setUserId(userId);
        work.setViewCount(0);
```

```
        work.setLikeCount(0);
        work.setStatus(1);

        userWorkMapper.insert(work);
        log.info("创建作品成功: userId={}, workId={}", userId,
work.getId());

        return work.getId();
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public void updateWork(UserWorkDTO userWorkDTO, Long userId) {
        UserWork existingWork =
userWorkMapper.selectById(userWorkDTO.getId());
        if (existingWork == null) {
            throw new BusinessException("作品不存在");
        }
        if (!existingWork.getUserId().equals(userId)) {
            throw new BusinessException("无权限修改此作品");
        }

        UserWork work = BeanUtil.copyProperties(userWorkDTO,
UserWork.class);
        work.setUserId(userId);

        userWorkMapper.updateById(work);
        log.info("更新作品成功: userId={}, workId={}", userId,
work.getId());
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public void deleteWork(Long id, Long userId) {
        UserWork work = userWorkMapper.selectById(id);
        if (work == null) {
            throw new BusinessException("作品不存在");
        }
        if (!work.getUserId().equals(userId)) {
            throw new BusinessException("无权限删除此作品");
        }

        userWorkMapper.deleteById(id);
        log.info("删除作品成功: userId={}, workId={}", userId, id);
```

```
    }

    @Override
    public void incrementViewCount(Long id) {
        userWorkMapper.update(null,
                new LambdaUpdateWrapper<UserWork>()
                        .setSql("view_count = view_count + 1")
                        .eq(UserWork::getId, id)
        );
    }

    @Override
    public void incrementLikeCount(Long id) {
        userWorkMapper.update(null,
                new LambdaUpdateWrapper<UserWork>()
                        .setSql("like_count = like_count + 1")
                        .eq(UserWork::getId, id)
        );
    }

    private UserWorkVO convertToVO(UserWork work) {
        UserWorkVO vo = BeanUtil.copyProperties(work,
UserWorkVO.class);

        User user = userMapper.selectById(work.getUserId());
        if (user != null) {
            vo.setUserNickname(user.getNickname() != null ?
user.getNickname() : user.getUsername());
            vo.setUserAvatar(user.getAvatar());
        }

        if (StrUtil.isNotEmpty(work.getTags())) {
            vo.setTags(work.getTags().split(","));
        }
        vo.setIsPublic(work.getIsPublic() != null &&
work.getIsPublic() == 1);
        vo.setIsFeatured(work.getIsFeatured() != null &&
work.getIsFeatured() == 1);

        return vo;
    }
}
```

```
// ========== FILE:
backend\src\main\java\com\cultural\service\LikeService.java
==========
package com.cultural.service;

public interface LikeService {

    boolean toggleLike(Long userId, String targetType, Long
targetId);

    boolean isLiked(Long userId, String targetType, Long targetId);
}

// ========== FILE:
backend\src\main\java\com\cultural\service\NewsService.java
==========
package com.cultural.service;

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.cultural.dto.PageDTO;
import com.cultural.vo.NewsVO;

public interface NewsService {

    Page<NewsVO> getPageList(PageDTO pageDTO);

    NewsVO getDetailById(Long id);

    void incrementViewCount(Long id);
}

// ========== FILE:
backend\src\main\java\com\cultural\service\SearchService.java
==========
package com.cultural.service;

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.cultural.dto.SearchDTO;
import com.cultural.vo.SearchResultVO;

import java.util.List;

public interface SearchService {
```

```
    Page<SearchResultVO> search(SearchDTO searchDTO, Long userId);

    List<String> getHotKeywords(int limit);

    List<String> getUserSearchHistory(Long userId, int limit);

    void clearUserSearchHistory(Long userId);
}

// ========== FILE:
backend\src\main\java\com\cultural\service\TraditionalArtService.java
==========
package com.cultural.service;

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.cultural.dto.PageDTO;
import com.cultural.vo.TraditionalArtVO;

public interface TraditionalArtService {

    Page<TraditionalArtVO> getPageList(PageDTO pageDTO);

    TraditionalArtVO getDetailById(Long id);

    void incrementViewCount(Long id);
}

// ========== FILE:
backend\src\main\java\com\cultural\service\UserWorkService.java
==========
package com.cultural.service;

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.cultural.dto.PageDTO;
import com.cultural.dto.UserWorkDTO;
import com.cultural.vo.UserWorkVO;

public interface UserWorkService {

    Page<UserWorkVO> getPageList(PageDTO pageDTO);

    Page<UserWorkVO> getMyWorks(PageDTO pageDTO, Long userId);

    UserWorkVO getDetailById(Long id);
```

```
    Long createWork(UserWorkDTO userWorkDTO, Long userId);

    void updateWork(UserWorkDTO userWorkDTO, Long userId);

    void deleteWork(Long id, Long userId);

    void incrementViewCount(Long id);

    void incrementLikeCount(Long id);
}

// ========== FILE:
backend\src\main\java\com\cultural\vo\CategoryVO.java ==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

import java.time.LocalDateTime;

@Data
@Schema(description = "分类 VO")
public class CategoryVO {

    @Schema(description = "分类 ID")
    private Long id;

    @Schema(description = "分类名称")
    private String name;

    @Schema(description = "分类编码")
    private String code;

    @Schema(description = "分类类型")
    private String type;

    @Schema(description = "父分类 ID")
    private Long parentId;

    @Schema(description = "排序序号")
    private Integer sortOrder;

    @Schema(description = "分类图标")
```

```
    private String icon;

    @Schema(description = "分类描述")
    private String description;
}

// ========== FILE:
backend\src\main\java\com\cultural\vo\CelebrityVO.java ==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

import java.time.LocalDateTime;

@Data
@Schema(description = "历史名人 VO")
public class CelebrityVO {

    @Schema(description = "ID")
    private Long id;

    @Schema(description = "姓名")
    private String name;

    @Schema(description = "朝代")
    private String dynasty;

    @Schema(description = "朝代排序")
    private Integer dynastyOrder;

    @Schema(description = "城市 ID")
    private Long cityId;

    @Schema(description = "籍贯城市")
    private String city;

    @Schema(description = "头衔/称号")
    private String title;

    @Schema(description = "简介")
    private String description;

    @Schema(description = "详细介绍")
```

```
    private String content;

    @Schema(description = "头像")
    private String avatar;

    @Schema(description = "图片列表")
    private String[] images;

    @Schema(description = "出生年份")
    private String birthYear;

    @Schema(description = "逝世年份")
    private String deathYear;

    @Schema(description = "主要成就")
    private String[] achievements;

    @Schema(description = "代表作品")
    private String[] works;

    @Schema(description = "标签")
    private String[] tags;

    @Schema(description = "浏览次数")
    private Integer viewCount;

    @Schema(description = "点赞次数")
    private Integer likeCount;

    @Schema(description = "收藏次数")
    private Integer collectCount;

    @Schema(description = "评论次数")
    private Integer commentCount;

    @Schema(description = "是否已收藏")
    private Boolean isCollected;

    @Schema(description = "是否已点赞")
    private Boolean isLiked;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;
}
```

```
// ========== FILE: backend\src\main\java\com\cultural\vo\CityVO.java ==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

import java.time.LocalDateTime;

@Data
@Schema(description = "城市 VO")
public class CityVO {

    @Schema(description = "城市 ID")
    private Long id;

    @Schema(description = "城市名称")
    private String name;

    @Schema(description = "城市编码")
    private String code;

    @Schema(description = "所属省份")
    private String province;

    @Schema(description = "城市拼音")
    private String pinyin;

    @Schema(description = "首字母")
    private String initial;

    @Schema(description = "排序序号")
    private Integer sortOrder;

    @Schema(description = "城市简介")
    private String description;
}

// ========== FILE: backend\src\main\java\com\cultural\vo\CommentVO.java ==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
```

```
import lombok.Data;

import java.time.LocalDateTime;

@Data
@Schema(description = "评论 VO")
public class CommentVO {

    @Schema(description = "评论 ID")
    private Long id;

    @Schema(description = "用户 ID")
    private Long userId;

    @Schema(description = "用户昵称")
    private String userNickname;

    @Schema(description = "用户头像")
    private String userAvatar;

    @Schema(description = "目标类型")
    private String targetType;

    @Schema(description = "目标 ID")
    private Long targetId;

    @Schema(description = "父评论 ID")
    private Long parentId;

    @Schema(description = "回复用户 ID")
    private Long replyUserId;

    @Schema(description = "回复用户昵称")
    private String replyUserNickname;

    @Schema(description = "评论内容")
    private String content;

    @Schema(description = "评论图片")
    private String[] images;

    @Schema(description = "点赞数")
    private Integer likeCount;
```

```
    @Schema(description = "是否已点赞")
    private Boolean isLiked;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;
}

// ========== FILE:
backend\src\main\java\com\cultural\vo\HeritageVO.java ==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

import java.time.LocalDateTime;

@Data
@Schema(description = "非物质文化遗产 VO")
public class HeritageVO {

    @Schema(description = "ID")
    private Long id;

    @Schema(description = "名称")
    private String name;

    @Schema(description = "分类 ID")
    private Long categoryId;

    @Schema(description = "分类名称")
    private String category;

    @Schema(description = "城市 ID")
    private Long cityId;

    @Schema(description = "城市名称")
    private String city;

    @Schema(description = "简介")
    private String description;

    @Schema(description = "详细内容")
    private String content;
```

```
@Schema(description = "封面图片")
private String coverImage;

@Schema(description = "图片列表")
private String[] images;

@Schema(description = "视频链接")
private String videoUrl;

@Schema(description = "级别：1-国家级，2-省级，3-市级，4-县级")
private Integer level;

@Schema(description = "级别名称")
private String levelName;

@Schema(description = "入选年份")
private String heritageYear;

@Schema(description = "申报单位")
private String declarer;

@Schema(description = "代表性传承人")
private String inheritor;

@Schema(description = "标签")
private String[] tags;

@Schema(description = "浏览次数")
private Integer viewCount;

@Schema(description = "点赞次数")
private Integer likeCount;

@Schema(description = "收藏次数")
private Integer collectCount;

@Schema(description = "评论次数")
private Integer commentCount;

@Schema(description = "是否已收藏")
private Boolean isCollected;

@Schema(description = "是否已点赞")
private Boolean isLiked;
```

```
    @Schema(description = "创建时间")
    private LocalDateTime createTime;
}

// ========== FILE: backend\src\main\java\com\cultural\vo\NewsVO.java ==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

import java.time.LocalDateTime;

@Data
@Schema(description = "新闻资讯 VO")
public class NewsVO {

    @Schema(description = "ID")
    private Long id;

    @Schema(description = "标题")
    private String title;

    @Schema(description = "分类 ID")
    private Long categoryId;

    @Schema(description = "分类名称")
    private String category;

    @Schema(description = "摘要")
    private String summary;

    @Schema(description = "内容")
    private String content;

    @Schema(description = "封面图片")
    private String coverImage;

    @Schema(description = "图片列表")
    private String[] images;

    @Schema(description = "作者")
    private String author;
```

```
    @Schema(description = "作者 ID")
    private Long authorId;

    @Schema(description = "来源")
    private String source;

    @Schema(description = "来源链接")
    private String sourceUrl;

    @Schema(description = "标签")
    private String[] tags;

    @Schema(description = "浏览次数")
    private Integer viewCount;

    @Schema(description = "点赞次数")
    private Integer likeCount;

    @Schema(description = "收藏次数")
    private Integer collectCount;

    @Schema(description = "评论次数")
    private Integer commentCount;

    @Schema(description = "是否置顶")
    private Boolean isTop;

    @Schema(description = "是否热门")
    private Boolean isHot;

    @Schema(description = "是否已收藏")
    private Boolean isCollected;

    @Schema(description = "是否已点赞")
    private Boolean isLiked;

    @Schema(description = "发布时间")
    private LocalDateTime publishTime;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;
}
```

```
// ========== FILE:
backend\src\main\java\com\cultural\vo\SearchResultVO.java ==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

import java.time.LocalDateTime;

@Data
@Schema(description = "搜索结果 VO")
public class SearchResultVO {

    @Schema(description = "数据 ID")
    private Long id;

    @Schema(description = "数据类型：
heritage/art/celebrity/news/work")
    private String type;

    @Schema(description = "标题/名称")
    private String title;

    @Schema(description = "描述/摘要")
    private String description;

    @Schema(description = "封面图片")
    private String coverImage;

    @Schema(description = "分类")
    private String category;

    @Schema(description = "城市")
    private String city;

    @Schema(description = "浏览次数")
    private Integer viewCount;

    @Schema(description = "点赞次数")
    private Integer likeCount;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;
}
```

```
// ========== FILE:
backend\src\main\java\com\cultural\vo\TokenVO.java ==========
package com.cultural.vo;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class TokenVO {

    private String token;

    private String tokenType = "Bearer";

    private Long expiresIn;

    private UserVO userInfo;

    public TokenVO(String token, Long expiresIn, UserVO userInfo) {
        this.token = token;
        this.expiresIn = expiresIn;
        this.userInfo = userInfo;
    }
}

// ========== FILE:
backend\src\main\java\com\cultural\vo\TraditionalArtVO.java
==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

import java.time.LocalDateTime;

@Data
@Schema(description = "传统艺术 VO")
public class TraditionalArtVO {

    @Schema(description = "ID")
```

```
private Long id;

@Schema(description = "名称")
private String name;

@Schema(description = "分类 ID")
private Long categoryId;

@Schema(description = "分类名称")
private String category;

@Schema(description = "城市 ID")
private Long cityId;

@Schema(description = "城市名称")
private String city;

@Schema(description = "简介")
private String description;

@Schema(description = "详细介绍")
private String content;

@Schema(description = "封面图片")
private String coverImage;

@Schema(description = "图片列表")
private String[] images;

@Schema(description = "视频链接")
private String videoUrl;

@Schema(description = "起源时期")
private String originPeriod;

@Schema(description = "代表艺术家")
private String artist;

@Schema(description = "标签")
private String[] tags;

@Schema(description = "浏览次数")
private Integer viewCount;
```

```
    @Schema(description = "点赞次数")
    private Integer likeCount;

    @Schema(description = "收藏次数")
    private Integer collectCount;

    @Schema(description = "评论次数")
    private Integer commentCount;

    @Schema(description = "是否已收藏")
    private Boolean isCollected;

    @Schema(description = "是否已点赞")
    private Boolean isLiked;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;
}

// ========== FILE: backend\src\main\java\com\cultural\vo\UserVO.java
==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

import java.time.LocalDate;
import java.time.LocalDateTime;

@Data
@Schema(description = "用户信息 VO")
public class UserVO {

    @Schema(description = "用户 ID")
    private Long id;

    @Schema(description = "用户名")
    private String username;

    @Schema(description = "昵称")
    private String nickname;

    @Schema(description = "真实姓名")
    private String realName;
```

```
@Schema(description = "邮箱")
private String email;

@Schema(description = "手机号")
private String phone;

@Schema(description = "头像URL")
private String avatar;

@Schema(description = "性别：0-未知，1-男，2-女")
private Integer gender;

@Schema(description = "性别名称")
private String genderName;

@Schema(description = "出生日期")
private LocalDate birthday;

@Schema(description = "省份")
private String province;

@Schema(description = "城市")
private String city;

@Schema(description = "个人简介")
private String bio;

@Schema(description = "兴趣标签")
private String[] interests;

@Schema(description = "角色")
private String role;

@Schema(description = "作品数量")
private Integer workCount;

@Schema(description = "收藏数量")
private Integer collectCount;

@Schema(description = "获赞数量")
private Integer likeCount;

@Schema(description = "创建时间")
```

```
    private LocalDateTime createTime;
}

// ========== FILE:
backend\src\main\java\com\cultural\vo\UserWorkVO.java ==========
package com.cultural.vo;

import io.swagger.v3.oas.annotations.media.Schema;
import lombok.Data;

import java.time.LocalDateTime;

@Data
@Schema(description = "用户作品 VO")
public class UserWorkVO {

    @Schema(description = "ID")
    private Long id;

    @Schema(description = "用户 ID")
    private Long userId;

    @Schema(description = "用户昵称")
    private String userNickname;

    @Schema(description = "用户头像")
    private String userAvatar;

    @Schema(description = "标题")
    private String title;

    @Schema(description = "描述")
    private String description;

    @Schema(description = "图片 URL")
    private String imageUrl;

    @Schema(description = "缩略图 URL")
    private String thumbnailUrl;

    @Schema(description = "AI 生成提示词")
    private String prompt;

    @Schema(description = "风格")
```

```
    private String style;

    @Schema(description = "文化元素")
    private String culturalElement;

    @Schema(description = "关联非遗 ID")
    private Long heritageId;

    @Schema(description = "关联艺术 ID")
    private Long artId;

    @Schema(description = "标签")
    private String[] tags;

    @Schema(description = "浏览次数")
    private Integer viewCount;

    @Schema(description = "点赞次数")
    private Integer likeCount;

    @Schema(description = "收藏次数")
    private Integer collectCount;

    @Schema(description = "评论次数")
    private Integer commentCount;

    @Schema(description = "是否公开")
    private Boolean isPublic;

    @Schema(description = "是否精选")
    private Boolean isFeatured;

    @Schema(description = "是否已收藏")
    private Boolean isCollected;

    @Schema(description = "是否已点赞")
    private Boolean isLiked;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;
}
```